

Beni Suef National University
Faculty of Computer Science & Information Technology

PROJECT REPORT

Tic Tac Toe

AI-Powered Game with Minimax Algorithm

Course	Introduction to Artificial Intelligence
Supervisor	Dr. Noha
Teaching Assistant	Eng. Abd Elrahman
Academic Year	2025 – 2026

Team Members

#	Student Name
1	Omar Ragab
2	Mohammed Ramadan
3	Hazem Gamal

1. Introduction

Tic Tac Toe is one of the most well-known two-player strategy games. Despite its simplicity, it serves as an excellent case study for understanding fundamental concepts in Artificial Intelligence, including game-tree search, adversarial reasoning, and decision-making under constraints.

This project presents a fully functional, AI-powered Tic Tac Toe application developed in Python using the Tkinter GUI library. The application supports multiple board sizes (3×3, 4×4, and 5×5), three difficulty levels, and both Player vs. AI and Player vs. Player game modes. The AI engine is powered by the Minimax algorithm, a classic adversarial search technique.

UI Application Main Menu



Figure 1: The Tic Tac Toe application main menu showing all configuration options available to the player before starting a game.

Purpose: Shows the entry point of the application and gives the reader an immediate visual understanding of what the program looks like when first launched.

2. Project Overview

2.1 Objectives

The primary objectives of this project are:

- Implement a playable Tic Tac Toe game with a professional graphical interface.
- Integrate an AI opponent capable of making optimal and sub-optimal moves based on a configurable difficulty level.
- Support variable board sizes to increase game complexity and replayability.
- Provide a smooth, intuitive user experience with score tracking across sessions.

2.2 Technologies Used

Technology	Version / Detail	Purpose
Python	3.x	Core programming language
Tkinter	Built-in	GUI / graphical interface
Random module	Built-in	Easy & medium AI moves
math.inf	Built-in	Minimax scoring bounds

3. System Design & Architecture

The application is organized into three main layers: the Game Logic layer, the AI Engine layer, and the GUI layer. This separation of concerns ensures maintainability and clarity.

3.1 Module Breakdown

Module / Function	Responsibility
<code>generate_win_ways(n)</code>	Dynamically generates all winning combinations for an $n \times n$ board, including rows, columns, and both diagonals.
<code>winner(b, win_ways)</code>	Scans the board against all winning combinations to detect if a player has won and returns the winning line.
<code>full(b)</code>	Checks whether every cell on the board is occupied, signalling a draw condition.
<code>minimax(...)</code>	Recursive adversarial search that evaluates all possible game states and returns the optimal score and move index.
<code>best_move(...)</code>	Entry point for AI move selection; delegates to minimax (hard), rule-based logic (medium), or random (easy).
<code>Game class</code>	Main GUI controller built on Tkinter; manages menus, board

rendering, click events, and score tracking.

3.2 Win-Way Generation Code

The `generate_win_ways(n)` function is the backbone of the win-detection system. It constructs all valid winning lines for any board size n at runtime, making the game fully generalisable without hard-coded winning combinations.

<> generate_win_ways(n) Function

```
player = 'X'
agent = 'O'
empty = ' '

# ===== LOGIC =====
def generate_win_ways(n):
    ways = []
    for r in range(n):
        ways.append(tuple(r*n + c for c in range(n)))
    for c in range(n):
        ways.append(tuple(r*n + c for r in range(n)))
    ways.append(tuple(i*n + i for i in range(n)))
    ways.append(tuple(i*n + (n-1-i) for i in range(n)))
    return ways
```

Figure 2: The `generate_win_ways(n)` function that dynamically builds all winning combinations (rows, columns, diagonals) for any board size n .

Purpose: Demonstrates the dynamic and generalisable approach to win detection, which is a key design decision that allows the game to support 3×3, 4×4, and 5×5 boards from a single implementation.

3.3 Winner Detection Code

The `winner(b, win_ways)` function iterates over all generated winning lines and checks if any line contains identical non-empty marks. It returns both the winning mark and the exact winning line indices for UI highlighting.

< winner() and full() Functions

```
def winner(b, win_ways):
    for way in win_ways:
        vals = [b[i] for i in way]
        if vals[0] != empty and all(v == vals[0] for v in vals):
            return vals[0], way
    return None, None

def full(b): return all(c != empty for c in b)
```

Figure 3: The `winner()` and `full()` helper functions responsible for detecting a win condition and a draw condition respectively.

Purpose: Shows how the game evaluates the board state after every move, which is called by both the UI event handler and the Minimax algorithm itself.

4. AI Engine — Minimax Algorithm

The AI opponent is driven by the Minimax algorithm, a decision-making algorithm commonly used in two-player, zero-sum games. The algorithm assumes both players play optimally — the AI (Maximizer) attempts to maximise its score, while the human player (Minimizer) attempts to minimise the AI's score.

4.1 How Minimax Works

Starting from the current board state, Minimax recursively explores every possible future move. Terminal states are assigned a score:

- AI wins → +10 minus depth (rewards faster wins)
- Player wins → depth minus 10 (penalises faster losses)
- Draw → 0

The algorithm propagates these scores back up the game tree, and the AI selects the move that leads to the highest guaranteed score.

4.2 Minimax Implementation

minimax() Recursive Function

```
# ===== AI =====
def minimax(b, depth, maximizing, win_ways):
    w,_ = winner(b, win_ways)
    if w == agent: return 10-depth, None
    if w == player: return depth-10, None
    if full(b): return 0, None

    if maximizing:
        best, move = -inf, None
        for i in range(len(b)):
            if b[i]==empty:
                b[i]=agent
                score,_ = minimax(b, depth+1, False, win_ways)
                b[i]=empty
                if score>best:
                    best, move = score, i
        return best, move
    else:
        best, move = inf, None
        for i in range(len(b)):
            if b[i]==empty:
                b[i]=player
                score,_ = minimax(b, depth+1, True, win_ways)
                b[i]=empty
                if score<best:
                    best, move = score, i
        return best, move
```

Figure 4: The complete minimax() recursive function showing base-case terminal evaluation, the maximizing branch (AI), and the minimizing branch (human player).

Purpose: This is the core of the project's AI. Showing the full implementation demonstrates how the algorithm explores the game tree and selects the optimal move through recursion.

4.3 Best-Move Dispatcher

The `best_move()` function acts as a dispatcher that selects the AI strategy based on the configured difficulty level and board size.

< best_move() Function

```
def best_move(b, level, win_ways):
    moves = [i for i in range(len(b)) if b[i]==empty]

    if level == "easy":
        return random.choice(moves)

    if level == "medium":
        # win
        for i in moves:
            b[i]=agent
            if winner(b,win_ways)[0]==agent:
                b[i]=empty; return i
            b[i]=empty
        # block
        for i in moves:
            b[i]=player
            if winner(b,win_ways)[0]==player:
                b[i]=empty; return i
            b[i]=empty
        return random.choice(moves)

    # hard (3x3 only)
    if len(b)==9:
        return minimax(b,0,True,win_ways)[1]

    return random.choice(moves)
```

Figure 5: The `best_move()` dispatcher function showing how each difficulty level (Easy, Medium, Hard) maps to a different decision strategy.

Purpose: Illustrates the clean separation between difficulty levels and explains exactly how the game transitions from trivial random play to optimal Minimax-based play.

4.4 Difficulty Levels Summary

Level	Behaviour
Easy	The AI selects a random valid cell. No strategic logic is applied, making it simple for beginners.
Medium	The AI first checks if it can win immediately; if not, it blocks the player from winning; otherwise it plays randomly. Provides a moderate challenge.
Hard	On a 3×3 board the full Minimax algorithm is applied, making the AI unbeatable. On larger boards, random moves are used due to the exponential search space.

5. Game Features & GUI

5.1 Core Features

- Variable board sizes: 3×3, 4×4, and 5×5 grids.
- Three AI difficulty levels: Easy, Medium, and Hard.
- Player vs. AI mode and two-player (PvP) mode.
- Option to let the AI make the first move.
- Persistent score tracking across multiple rounds (X wins, O wins, Draws).
- Winning cells are highlighted in green at the end of each game.
- Clean main menu with mode, difficulty, and board-size selection.

5.2 GUI Source Code

<> Game Class — __init__ and build_menu() Methods


```

# ===== GUI =====
class Game:
    def __init__(self, root):
        self.root = root
        self.root.title("Tic Tac Toe ULTRA")
        self.root.geometry("520x720")
        self.root.configure(bg="#0f172a")

        self.size = 3
        self.mode = "AI"
        self.level = "medium"
        self.level_var = tk.StringVar(value=self.level)
        self.ai_first = tk.BooleanVar(value=False)

        self.score = {"X":0,"O":0,"D":0}

        self.build_menu()

    # ===== BUTTON =====
    def card(self, parent, text, cmd):
        btn = tk.Label(parent, text=text, font=("Segoe UI", 14, "bold"),
                        bg="#1e293b", fg="white", padx=20, pady=12, cursor="hand2")
        btn.bind("<Button-1>", lambda e: cmd())
        btn.bind("<Enter>", lambda e: btn.config(bg="#334155"))
        btn.bind("<Leave>", lambda e: btn.config(bg="#1e293b"))
        return btn

    # ===== MENU =====
    def build_menu(self):
        self.clear()

        tk.Label(self.root, text="Tic Tac Toe", font=("Segoe UI", 28, "bold"),
                 bg="#0f172a", fg="#38bdf8").pack(pady=20)

        # Mode
        frame_mode = tk.Frame(self.root, bg="#0f172a")
        frame_mode.pack(pady=6)

        self.card(frame_mode, "Play vs AI", lambda: self.set_mode("AI")).grid(row=0, column=0, padx=5)
        self.card(frame_mode, "2 Players", lambda: self.set_mode("PVP")).grid(row=0, column=1, padx=5)

        # Difficulty
        tk.Label(self.root, text="Difficulty", bg="#0f172a", fg="white").pack(pady=8)

        frame_lvl = tk.Frame(self.root, bg="#0f172a")
        frame_lvl.pack()

        for i, lvl in enumerate(["easy", "medium", "hard"]):
            rb = tk.Radiobutton(frame_lvl, text=lvl.capitalize(),
                               variable=self.level_var, value=lvl,
                               command=lambda l=lvl: self.set_level(l),
                               bg="#0f172a", fg="white", selectcolor="#1e293b")
            rb.grid(row=0, column=i, padx=10)

        # Size
        tk.Label(self.root, text="Board Size", bg="#0f172a", fg="white").pack(pady=8)

        frame_size = tk.Frame(self.root, bg="#0f172a")
        frame_size.pack()

        for i, s in enumerate([3, 4, 5]):
            self.card(frame_size, f"{s} x {s}", lambda n=s: self.set_size(n)).grid(row=0, column=i, padx=5)

        # AI start
        tk.Checkbutton(self.root, text="AI Starts", variable=self.ai_first,
                      bg="#0f172a", fg="white", selectcolor="#1e293b").pack(pady=10)

        # Start button
        start_btn = tk.Label(self.root, text="START GAME",
                             font=("Segoe UI", 16, "bold"),
                             bg="#22c55e", fg="black",
                             padx=25, pady=14, cursor="hand2")
        start_btn.pack(pady=20)
        start_btn.bind("<Button-1>", lambda e: self.start())

    def set_mode(self, m): self.mode = m
    def set_level(self, l): self.level = l
    def set_size(self, s): self.size = s

```

Figure 6: The Game class initialiser and the build_menu() method showing how the main menu is constructed using Tkinter Label, Radiobutton, Checkbutton, and Frame widgets.

Purpose: Shows the GUI initialisation code and explains how the configurable game settings (board size, mode, difficulty) are stored as instance variables and reflected in the interface.

< start(), click(), and move() Methods

```
# ===== GAME =====
def start(self):
    self.clear()

    self.board=[empty]*(self.size*self.size)
    self.win_ways=generate_win_ways(self.size)
    self.game_over=False
    self.locked=False
    self.current=player

    self.status=tk.Label(self.root,text="Player X Turn",bg="#0f172a",fg="white")
    self.status.pack()

    self.score_lbl=tk.Label(self.root,text=self.score_text(),
                            bg="#0f172a",fg="#94a3b8")
    self.score_lbl.pack()

    self.grid=tk.Frame(self.root,bg="#0f172a")
    self.grid.pack()

    self.btns=[]
    for i in range(len(self.board)):
        b=tk.Label(self.grid,text="",font=("Segoe UI",28,"bold"),
                  width=4,height=2,bg="#1e293b",fg="white",cursor="hand2")
        b.grid(row=i//self.size,column=i%self.size,padx=5,pady=5)

        b.bind("<Button-1>",lambda e,i=i:self.click(i))
        self.btns.append(b)

    tk.Button(self.root,text="Back",command=self.build_menu).pack(pady=10)

    if self.mode=="AI" and self.ai_first.get():
        self.locked=True
        self.status.config(text="AI Thinking...")
        self.root.after(400,self.ai_move)

def score_text(self):
    return f"Score X:{self.score['X']} O:{self.score['O']} D:{self.score['D']}"

def click(self,i):
    if self.game_over or self.locked or self.board[i]!=empty: return

    self.move(i,self.current)
    if self.check(): return

    if self.mode=="AI":
        self.locked=True
        self.status.config(text="AI Thinking...")
        self.root.after(300,self.ai_move)
    else:
        self.current = agent if self.current==player else player
        self.status.config(text=f"Player {self.current} Turn")

def move(self,i,mark):
    self.board[i]=mark
    self.btns[i].config(text=mark,fg="#38bdf8" if mark=='X' else "#f87171")
```

Figure 7: The start(), click(), and move() methods that handle game initialisation, user input validation, and board state updates.

Purpose: Demonstrates the event-driven game loop: how the board is drawn, how human moves are validated and applied, and how the transition to the AI's turn is triggered.

<> ai_move() and check() Methods

```

def ai_move(self):
    i=best_move(self.board,self.level,self.win_ways)
    if i is not None:
        self.move(i,agent)

    if not self.check():
        self.locked=False
        self.status.config(text="Player X Turn")

def check(self):
    w,line=winner(self.board,self.win_ways)

    if w or full(self.board):
        self.game_over=True
        self.locked=True

        if w:
            for i in line:
                self.btns[i].config(bg="#22c55e")
            messagebox.showinfo("Game Over",f"{w} wins!")
            self.score[w]+=1
        else:
            messagebox.showinfo("Game Over", "Draw")
            self.score['D']+=1

        self.score_lbl.config(text=self.score_text())
        return True
    return False

def clear(self):
    for w in self.root.winfo_children():
        w.destroy()

# ===== RUN =====
root=tk.Tk()
Game(root)
root.mainloop()

```

Figure 8: The `ai_move()` and `check()` methods that drive the AI response loop and post-move state evaluation.

Purpose: Shows how the AI move is scheduled asynchronously using `root.after()`, preventing UI freezing, and how game-over conditions (win, draw) trigger score updates and the result dialog.

6. Game Flow & Screenshots

6.1 Active 3×3 Game Board

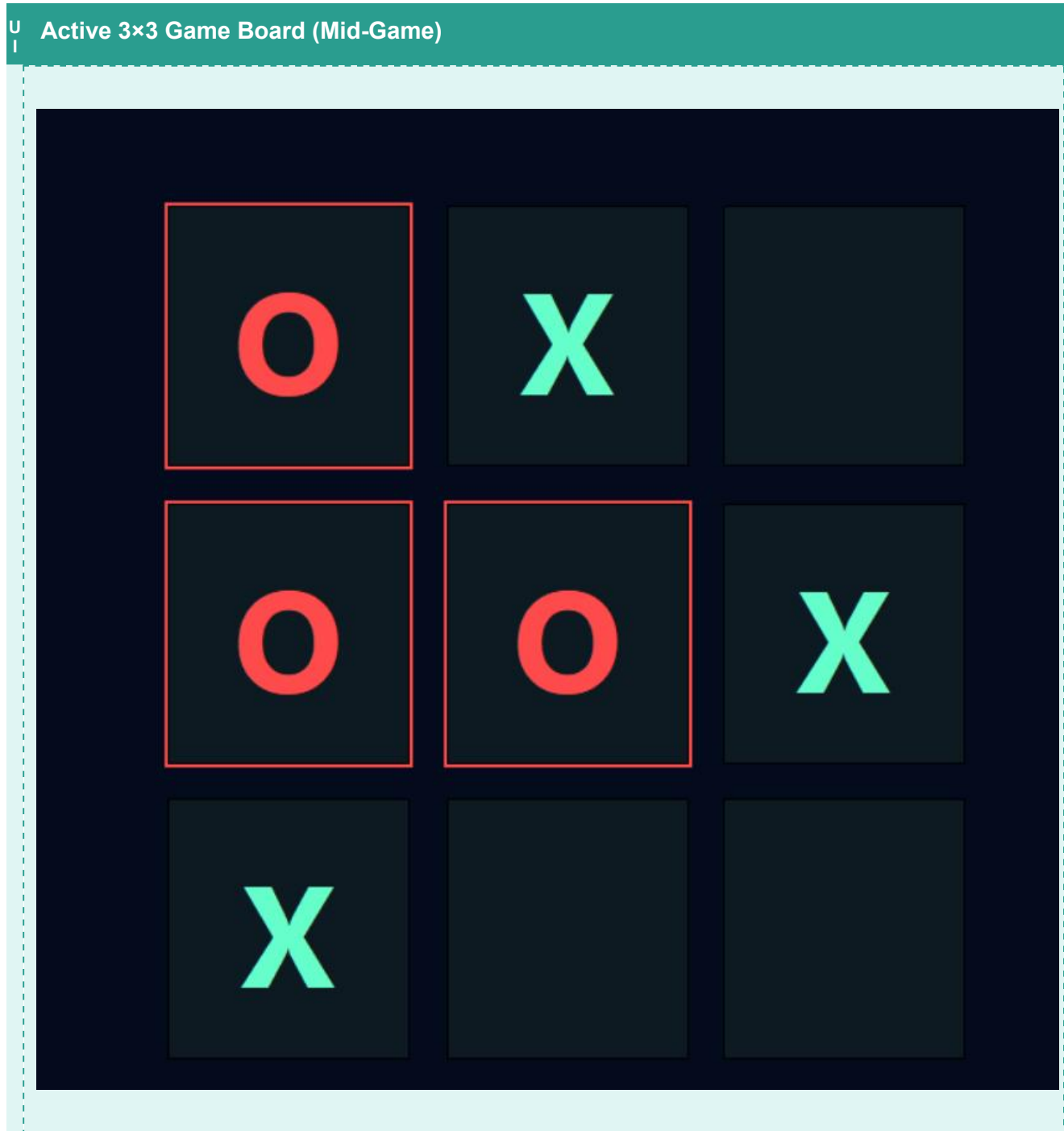


Figure 9: A 3×3 game board mid-game with both player (X) and AI (O) marks visible, along with the status bar and score display.

Purpose: Gives the reader a clear view of the game interface during active play, showing the colour-coded marks (blue X, red O), the dark theme, and the real-time status updates.

6.2 AI Thinking State

UI AI Thinking Status Label

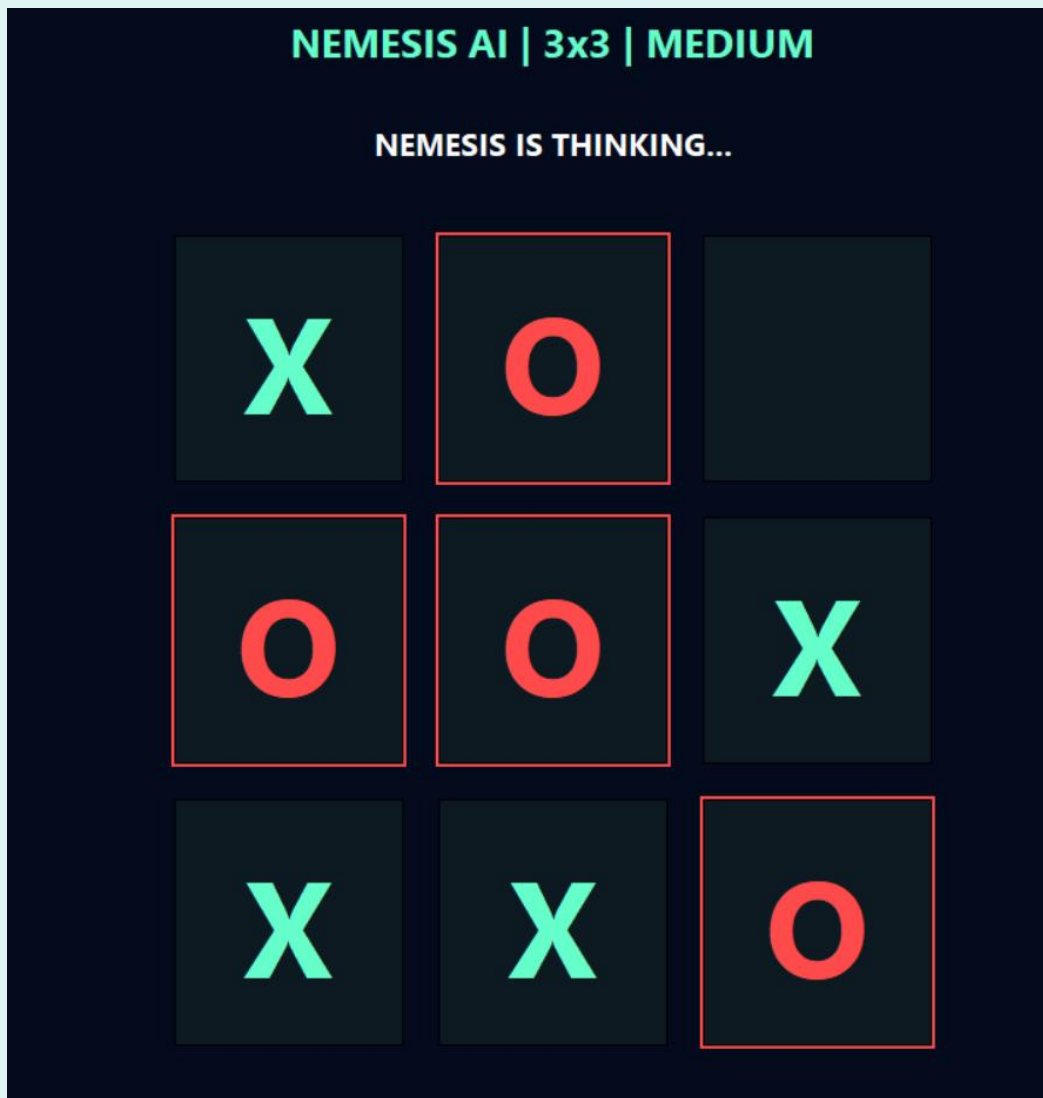


Figure 10: The game window showing the 'AI Thinking...' status message displayed while the application is computing the AI's next move.

Purpose: Demonstrates the asynchronous move scheduling that prevents the UI from freezing during AI computation, providing visual feedback to the user that the system is processing.

6.3 Winning State — Green Highlight

UI Winning State with Green-Highlighted Cells

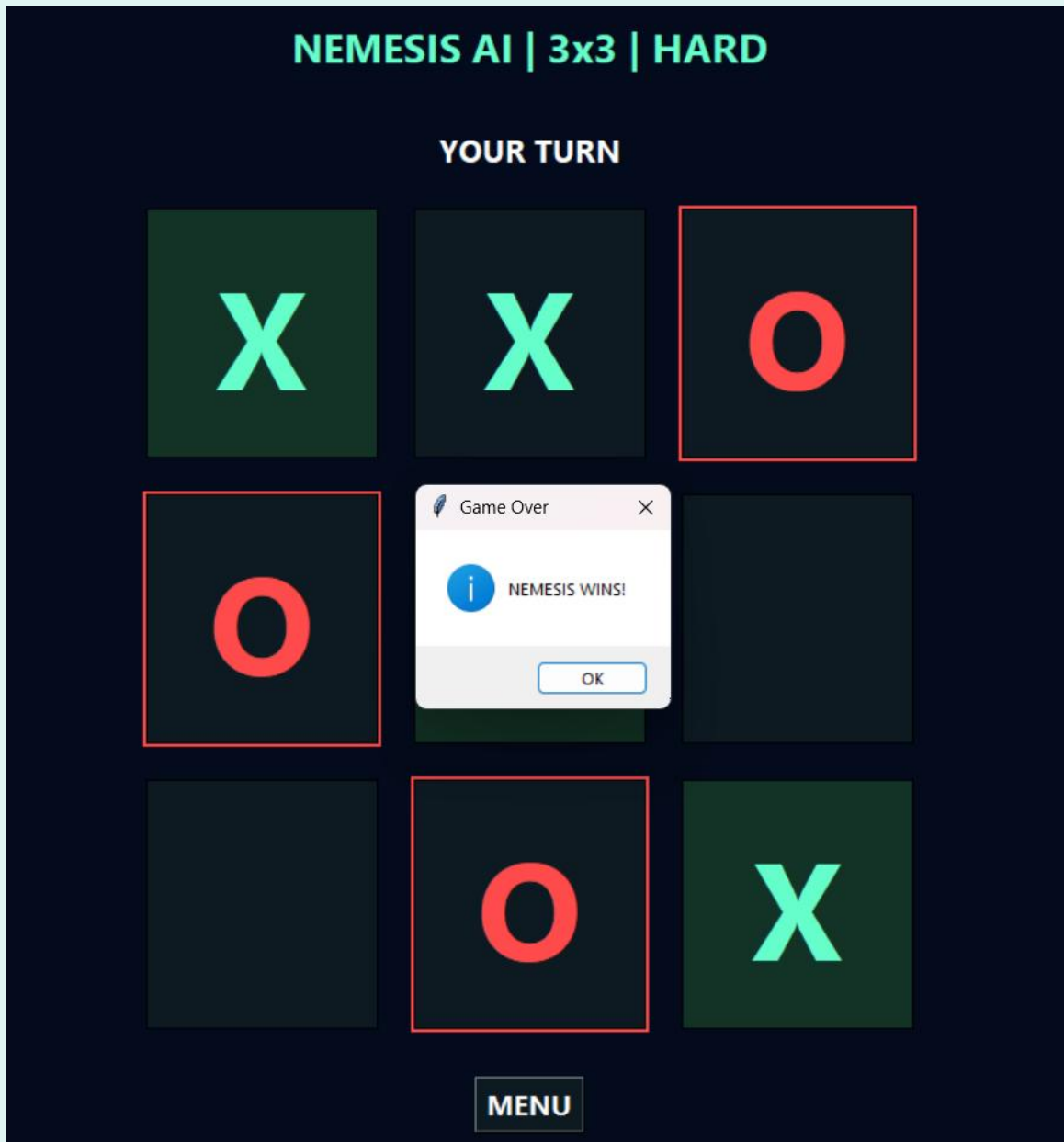


Figure 11: The game board at the moment a player wins, showing the three winning cells highlighted in green and the result announcement dialog box.

Purpose: Shows the win-detection visualisation in action, confirming that the correct winning line is identified and highlighted, and that the result is clearly communicated to the user.

6.4 Draw State

UI OUTPUT / UI SCREENSHOT #12 Draw / Tie Game State

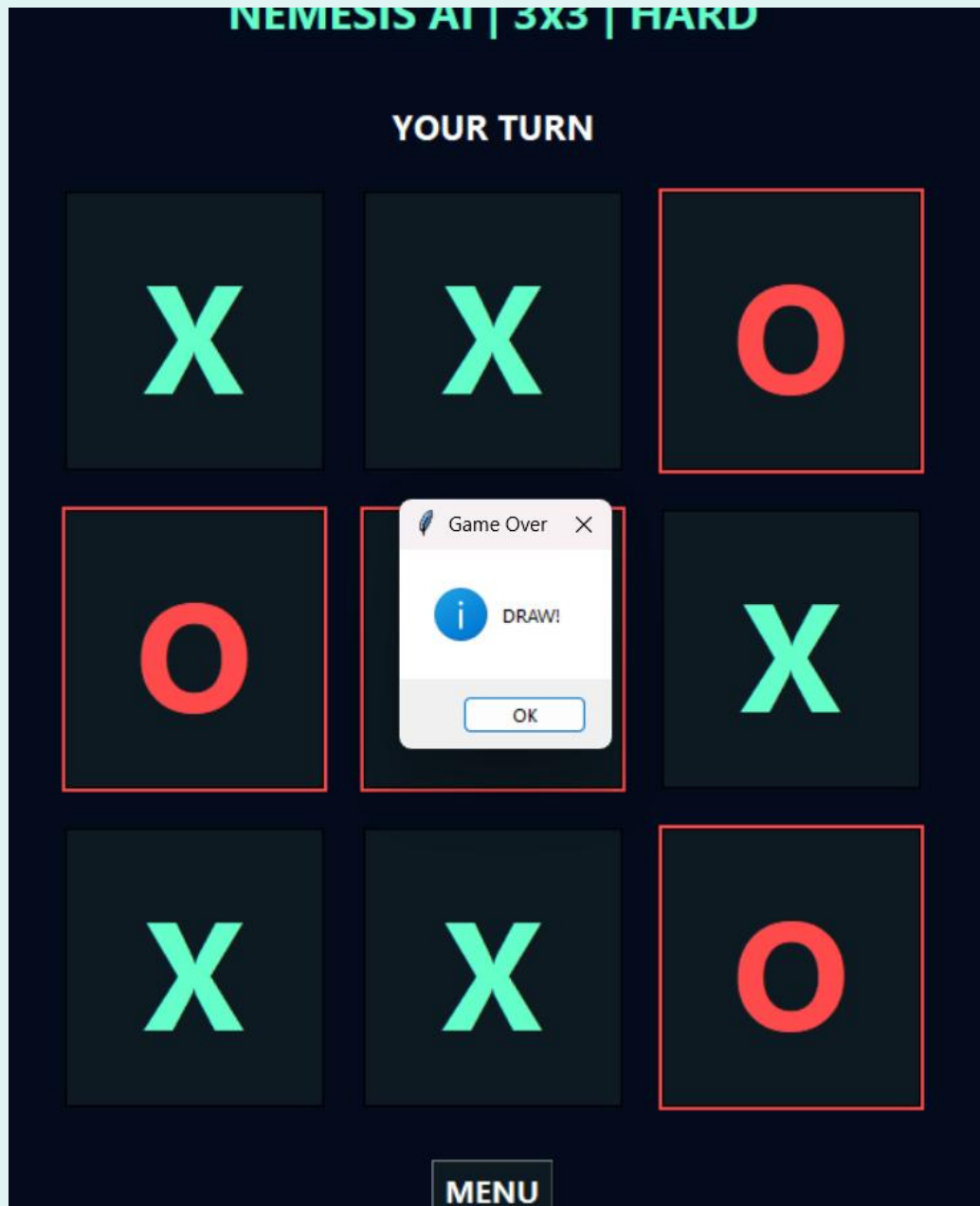


Figure 12: A completed 3×3 game board with all cells filled but no winner, and the draw announcement dialog.

Purpose: Demonstrates the draw-detection logic (the full() function) and confirms the correct handling of the

tie scenario, which is one of the most common outcomes in optimal 3×3 play.

6.5 Score Tracker

UI Score Tracker After Multiple Rounds

NEMESIS AI | 3x3 | HARD

NEMESIS IS THINKING...

Figure 13: The score tracker label displaying accumulated wins for X, wins for O, and draws across multiple completed rounds.

Purpose: Verifies that the score persistence feature works correctly across rounds and that all three outcome types (X win, O win, Draw) are tracked and displayed in real time.

7. Variable Board Sizes

The game dynamically generates the board grid and all winning combinations at runtime using the board size parameter n. This means the same codebase handles 3×3, 4×4, and 5×5 boards without any hardcoded logic.

7.1 4×4 Board in Play

UI 4×4 Game Board

NEMESIS AI | 4x4 | HARD

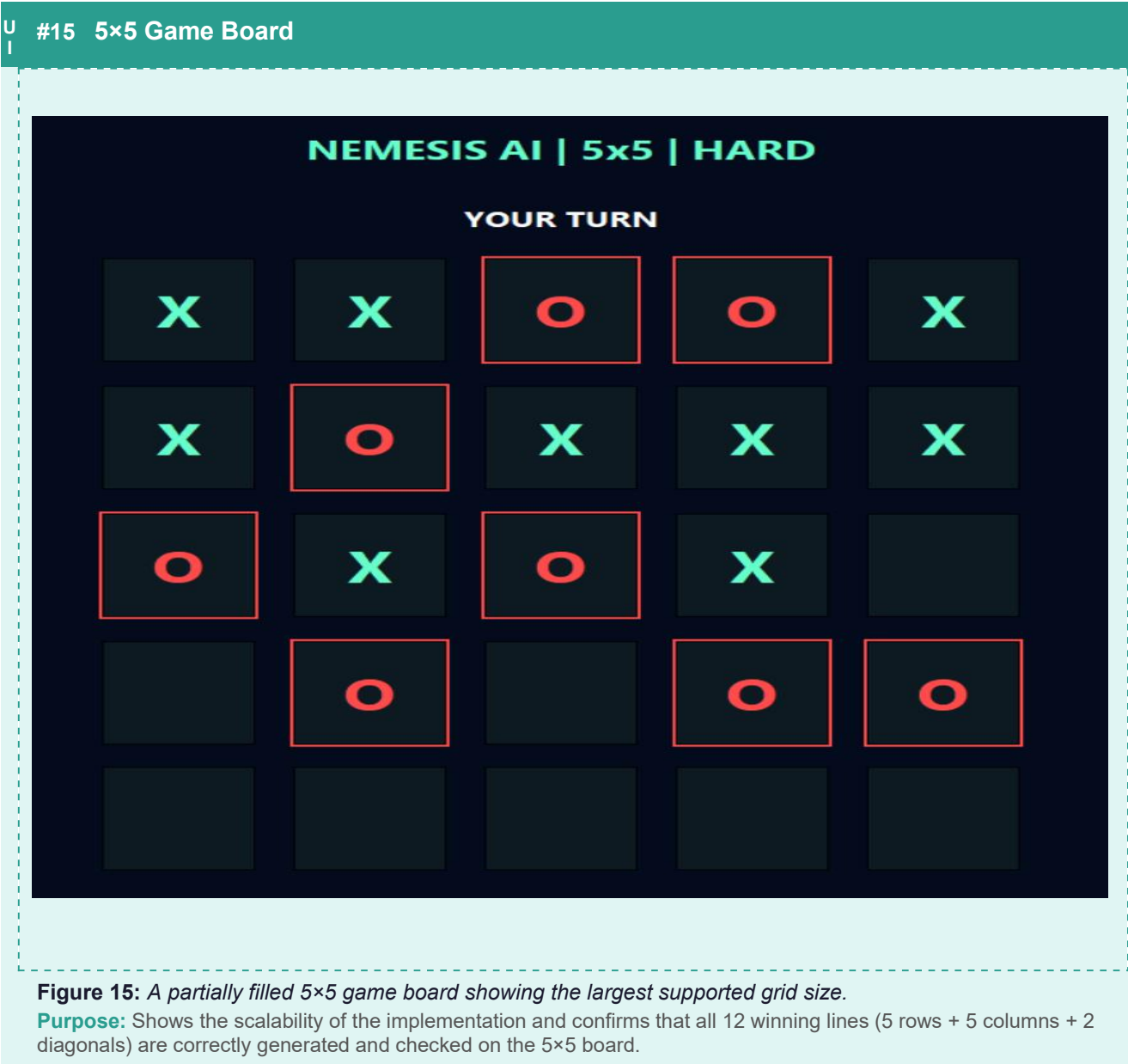
YOUR TURN

X	X	X	O
X	O	X	O
O	O	X	X
X	O	O	O

Figure 14: A partially filled 4×4 game board showing the larger grid layout supported by the application.

Purpose: Demonstrates the variable board size feature and confirms that the board rendering, win detection, and AI moves all work correctly on a 4×4 grid.

7.2 5×5 Board in Play



8. Challenges & Solutions

Challenge	Solution Adopted
Minimax exponential complexity on 4×4 / 5×5 boards	Minimax is restricted to 3×3 boards only. Larger boards use rule-based or random strategies to keep the game responsive.
Dynamic win-way generation for variable board sizes	Implemented generate_win_ways(n) to programmatically compute all winning lines rather than hard-coding them.
Preventing UI freeze during AI computation	Used root.after() to schedule AI moves asynchronously, keeping the Tkinter event loop responsive.
GUI state management (locked / game_over flags)	Introduced boolean flags (locked, game_over) to prevent race conditions between human clicks and AI responses.

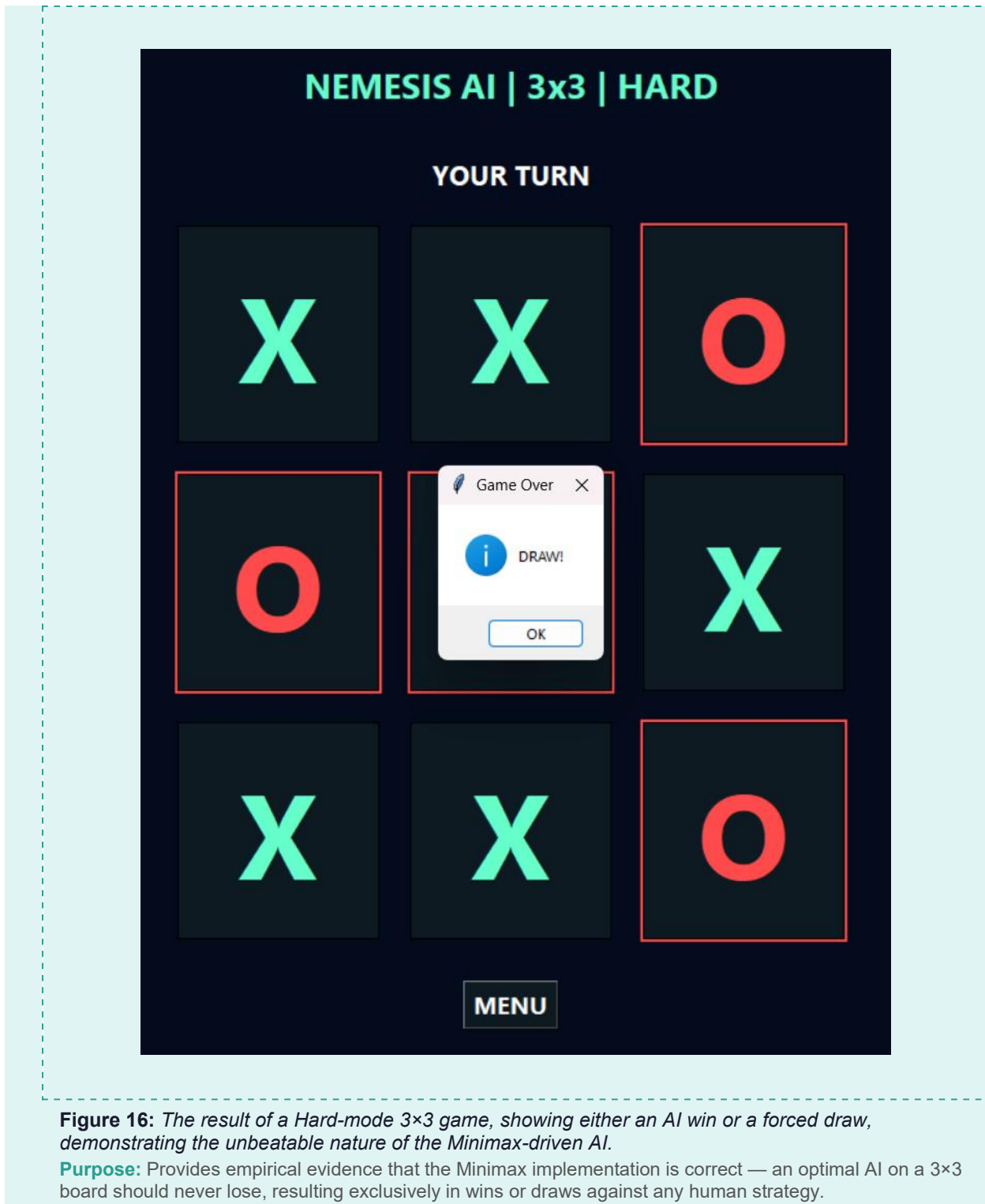
9. Testing & Results

9.1 Test Scenarios

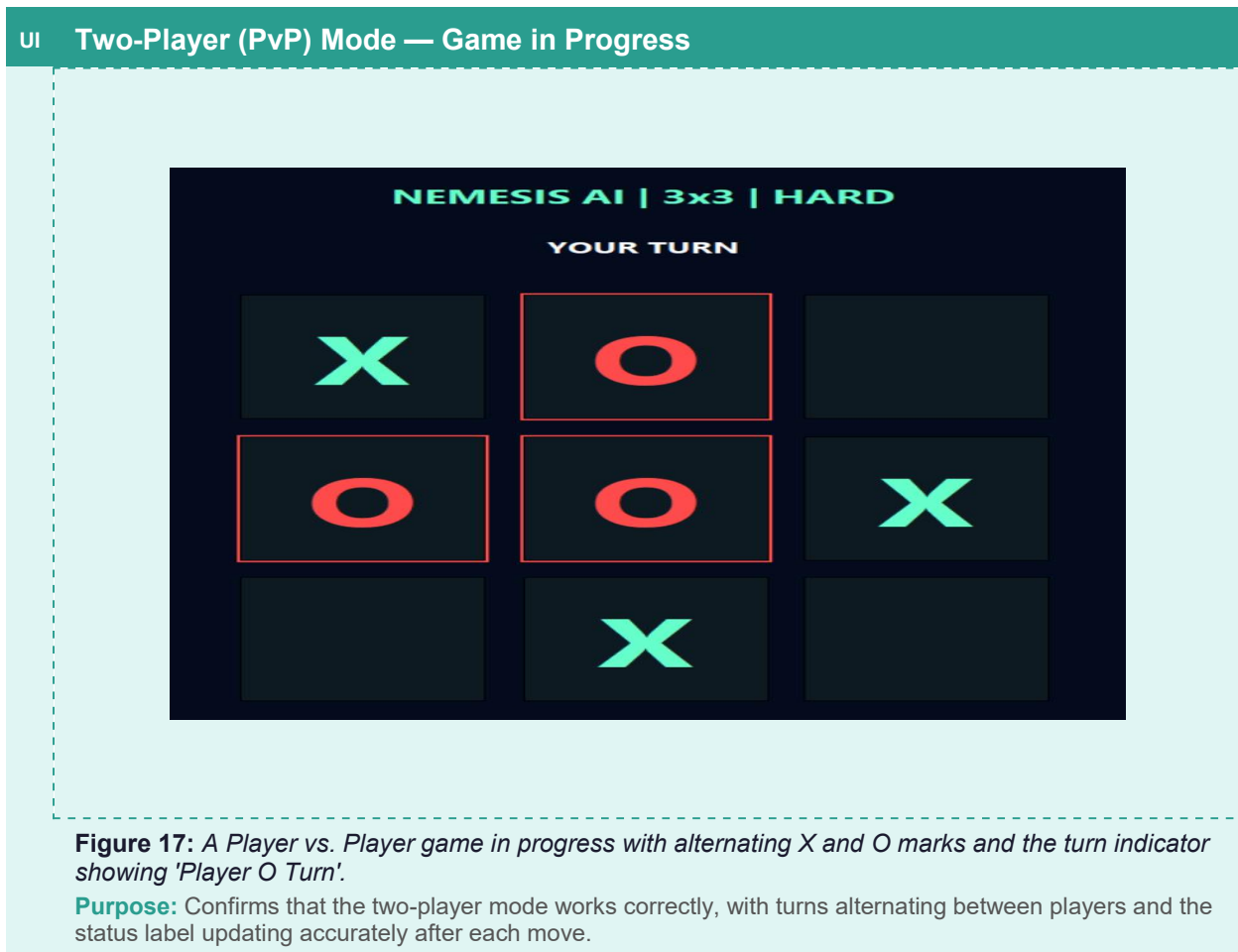
- 3×3 board — Hard mode: the AI consistently achieves a win or forces a draw; it never loses.
- 3×3 board — Easy mode: the AI behaves randomly, and a skilled human player can win.
- 3×3 board — Medium mode: the AI correctly blocks immediate winning threats.
- 4×4 and 5×5 boards: all winning combinations are detected correctly.
- Draw detection: when all cells are filled with no winner, a draw is correctly announced.
- AI-first option: the AI correctly makes the opening move before human interaction.
- Score persistence: scores accumulate correctly across multiple rounds.

9.2 Hard Mode — AI Unbeatable Proof

UI Hard Mode — Forced Draw Result



9.3 PvP Mode



10. Conclusion & Future Work

10.1 Conclusion

This project successfully demonstrates the integration of classical AI techniques into an interactive graphical application. The Minimax algorithm provides an optimal, unbeatable AI agent on the standard 3x3 board, while the rule-based medium difficulty and random easy difficulty cater to players of varying skill levels. The dynamic board generation and modular code structure make the application straightforward to understand and extend.

10.2 Future Improvements

- Alpha-Beta Pruning: reduce the Minimax search space to enable near-optimal play on 4x4 boards.
- Depth-limited Minimax with heuristic evaluation for larger boards.

- Online multiplayer support using Python sockets or a web framework.
- Animated move transitions and sound effects for a richer user experience.
- Undo / redo functionality to support game review and learning.